

etcdctl – (Backup, Security & Commands)

What is etcdctl?

- etcdctl is a command-line tool to interact with etcd database.
-  Think:
 -  “Hey etcd, save / read / backup data”
 -  etcdctl = CLI tool to manage and interact with etcd

Simple Analogy

- etcd =  Brain
- etcdctl =  Voice to talk to the brain

What Can You Do with etcdctl?

-  Backup & Restore etcd
-  Add / Remove cluster members
-  Get / Put / Delete keys
-  Check cluster health
-  Troubleshoot issues

TLS Security in etcdctl

Why TLS is Required?

- etcd is secure by default 
 -  Only trusted users can access it
 -  TLS ensures encrypted and authenticated communication with etcd

◆ 1. --cacert (CA Certificate)

- Verifies etcd server
- Trusts the Certificate Authority
-  Path:

`/etc/kubernetes/pki/etcd/ca.crt`

♦ 2. --cert (Client Certificate)

- Your identity proof (Client identity)
- 📁 Path:

`/etc/kubernetes/pki/etcd/server.crt`

♦ 3. --key (Private Key)

- Confirms you own the certificate (Proof of identity)
- 📁 Path:

`/etc/kubernetes/pki/etcd/server.key`

📁 File Locations (kubeadm)

```
/etc/kubernetes/pki/etcd/  
├─ ca.crt  
├─ server.crt  
└─ server.key
```

🔒 Why These Are Required?

- ✓ Secure communication
- ✓ Authentication
- ✓ Prevent unauthorized access

🔄 🔄 Secure Backup Command

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db \  
--endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt \  
--cert=/etc/kubernetes/pki/etcd/server.crt \  
--key=/etc/kubernetes/pki/etcd/server.key
```

🧠 What This Means

- ✓ "I trust your CA"
- ✓ "Here is my identity"
- ✓ "Let's communicate securely"

⚙️ Important Setup

```
export ETCDCTL_API=3
```

- 👉 Always use API version 3

🔍 Cluster Inspection Commands

📊 Check Cluster Health

```
ETCDCTL_API=3 etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
endpoint health
```

✓ Output

```
https://127.0.0.1:2379 is healthy
```

📋 List Cluster Members

```
etcdctl --write-out=table member list
```

Output:

ID	STATUS	NAME	PEER ADDRS	CLIENT
1234abcd...	started	node1	https://10.0.0.1:2380	https:



Check Snapshot Status

```
etcdctl --write-out=table snapshot status /tmp/etcd-snapshot.db
```

Output:

HASH	REVISION	TOTAL KEYS	TOTAL SIZE
fe4cf...	12345	1428	25 MB



Backup & Restore (Step-by-Step)

```
etcdctl snapshot save backup.db #
etcdctl snapshot restore backup.db --data-dir=/var/lib/etcd-restored #
etcdctl snapshot status backup.db #
```



Full Restore Scenario (kubeadm Cluster)

Situation You have snapshot:

```
/backup/etcd-snapshot.db
```

1 Stop Services

```
sudo systemctl stop kubelet
systemctl stop etcd
```

2 Backup Old Data

```
sudo mv /var/lib/etcd /var/lib/etcd.bak
```

3 Restore Snapshot

```
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db --data-dir /var
```

4 Fix Permissions

```
sudo chown -R etcd:etcd /var/lib/etcd
```

5 Start Services

```
sudo systemctl start kubelet
```

- ✓ etcd auto-starts (static pod)

6 Verify Health

```
etcdctl endpoint health
```

⚠ Important Checklist

 Step	 Action	 Why It Matters	 Pro Tip
 1	 Stop kubelet	 Prevents Kubernetes from interfering during restore	<code>systemctl stop kubelet</code>
 2	 Backup old data	 Safety fallback if restore fails	<code>Copy /var/lib/etcd</code>
 3	 Restore snapshot	 Rebuild etcd from backup	<code>Use etcdctl snapshot restore</code>
 4	 Fix permissions	 Ensure etcd can read/write data	<code>chown -R etcd:etcd</code>
 5	 Start kubelet	 Bring control plane back online	<code>systemctl start kubelet</code>
 6	 Verify health	 Confirm cluster is working	<code>etcdctl endpoint health</code>



GitOps vs etcd Backup

✓ GitOps (ArgoCD / Flux)

● GitOps Advantage

- With tools like Argo CD / Flux:

Git  → **Source** of truth
Cluster  → Recreated **from** Git

- ✓ No need to backup workloads
- Recreate cluster easily

! But etcd Still Matters

- Even with GitOps, etcd stores:
 -  Secrets
 -  ConfigMaps
 -  PVC metadata
 -  Tokens & certs
 - Cluster state



EKS vs kubeadm

 Feature	 Amazon Elastic Kubernetes Service (EKS)	 kubeadm (Self-managed)
 etcd Access	✗ Not accessible Less control EKS hides control plane	✓ Full access full control
 Backup	✓ AWS managed AWS handles snapshots	✗ Manual (etcdctl) you must manage
 High Availability	✓ Managed by AWS EKS provides HA control plane Reduced complexity	⚠ You configure
 Certificates	✗ Not accessible AWS manages PKI	✓ Available user-managed Certificates

 Feature	 Amazon Elastic Kubernetes Service (EKS)	 kubeadm (Self-managed)
 Responsibility	AWS Managed	You self-managed
 Control	Limited Less flexibility	Full full customization

Best Strategy : Production Setup

etcd **Snapshot**  (Control Plane)
 +
 Velero **Backup**  (Workloads + PVCs)
 ↓
 Complete **Disaster Recovery** 

Summary Table

- In etcdctl, `--cacert` is used to verify the server, `--cert` provides client identity, and `--key` is used for authentication, enabling secure TLS communication with etcd.

 Option	 Meaning	 What It Does	 File Path (kubeadm)
 <code>--cacert</code>	Trust server	 Verifies etcd server using CA	<code>/etc/kubernetes/pki/etcd/ca.crt</code>
 <code>--cert</code>	Identity	 Client certificate (who you are)	<code>/etc/kubernetes/pki/etcd/server.crt</code>
 <code>--key</code>	Authentication	 Private key for secure auth	<code>/etc/kubernetes/pki/etcd/server.key</code>

Common Mistakes

-  Forgetting TLS flags
-  Not setting `ETCDCTL_API=3`
-  Not backing up regularly
-  Losing cert files

Key Takeaways

- ✓ Use ETCDCTL_API=3
- ✓ Always use TLS flags
- ✓ Snapshots = Disaster recovery
- ✓ Never edit etcd manually

Final Summary

etcd 
↓
etcdctl  (communicate)
↓
TLS  (secure access)
↓
Snapshot  (backup)
↓
Restore  (recovery)

Ultimate One-Line

etcdctl securely manages etcd using TLS, and snapshots ensure full disaster recovery of Kubernetes clusters.

Kubernetes etcd – Complete Q&A Sheet

 Category	 Question / Topic	 Answer
 Basics	What is etcd?	Distributed key-value store used by Kubernetes
 Basics	Why Kubernetes uses etcd?	Strong consistency, HA, fast reads/writes
 Basics	Is etcd a database?	Yes (key-value, not relational)
 Basics	What data is stored?	Pods, Nodes, Secrets, ConfigMaps, Deployments
 Basics	Where is etcd located?	Control Plane node
 Architecture	Who talks to etcd?	Only kube-apiserver

 Category	 Question / Topic	 Answer
 Architecture	Direct access allowed?	 No
 Architecture	Protocol used?	gRPC over HTTP/2
 Architecture	Consistency algorithm?	RAFT
 Architecture	What is RAFT?	Leader-based consensus
 Architecture	Leader failure?	New leader elected automatically
 Deployment	Deployment types?	Stacked & External
 Deployment	Stacked etcd?	Runs on control plane node
 Deployment	External etcd?	Separate cluster
 Deployment	Best for production?	External etcd
 Security	Is etcd secure?	Yes (TLS enabled)
 Security	How secured?	TLS certificates
 Security	Access without cert?	 No
 Security	CA cert path	/etc/kubernetes/pki/etcd/ca.crt
 Security	Server cert path	/etc/kubernetes/pki/etcd/server.crt
 Security	Key path	/etc/kubernetes/pki/etcd/server.key
 Backup	Backup command?	etcdctl snapshot save
 Backup	Restore command?	etcdctl snapshot restore
 Backup	Why backup important?	etcd = cluster brain
 Backup	If etcd lost?	Entire cluster state lost
 Ports	Client port	2379
 Ports	Peer port	2380
 Storage	Data directory	/var/lib/etcd
 Tools	Tool used?	etcdctl

 Category	 Question / Topic	 Answer
 Tools	API version?	ETCDCTL_API=3
 Tools	Health check	<code>etcdctl endpoint health</code>
 Tools	List keys	<code>etcdctl get "" --prefix --keys-only</code>
 Scaling	Horizontally scalable?	Limited
 Scaling	Recommended nodes	3 or 5
 Scaling	Why odd number?	Maintain quorum
 Scaling	What is quorum?	Majority agreement
 Traps	Kubernetes without etcd?	 Not possible
 Traps	Edit etcd directly?	 Not recommended
 Traps	Is etcd stateless?	 No
 Traps	etcd vs ConfigMap	etcd = backend, ConfigMap = object
 Traps	etcd vs MySQL	KV store vs relational DB
 Scenarios	Pod not starting	Check etcd health
 Scenarios	Data missing	Restore snapshot
 Scenarios	API slow	Check etcd latency
 Summary	Key takeaway	"If etcd is down, Kubernetes is down."